

Compromised AI Cluster

Investigating a Ray Framework Compromise (CVE-2023-48022)

About the Author

Hi, I'm **Ashvin Acharya**, a cybersecurity enthusiast focused on **SOC Operations, Incident Response, Threat Hunting, and Malware Analysis**.

This report documents my investigation of a compromised Ray AI Cluster where an exposed Ray Job Submission API was abused to achieve **Remote Code Execution (RCE)**, establish a reverse shell, collect sensitive information, exfiltrate confidential data, and perform internal network reconnaissance.

I created this write-up to document my investigation process, demonstrate practical incident response skills, and share my learning with the cybersecurity community.

Connect with me

- LinkedIn:- <https://www.linkedin.com/in/ashvin-aacharya-a08553308>
 - GitHub: <https://github.com/0xAshvin>
 - X (Twitter): <https://x.com/0xAshvin>
-

Skills Demonstrated

- Incident Response
- Threat Hunting
- Network Traffic Analysis
- Wireshark
- Linux Investigation
- MITRE ATT&CK Mapping
- IOC Identification
- Timeline Reconstruction

Executive Summary

An unusual spike in traffic targeting a publicly exposed **Ray AI Cluster** triggered this investigation. Initial packet analysis quickly revealed that the traffic was not associated with legitimate AI workloads. Instead, the attacker was abusing Ray's **Job Submission API** to remotely execute commands on the server.

Further investigation confirmed exploitation of **CVE-2023-48022**, allowing unauthenticated command execution through the exposed Ray framework. After obtaining code execution, the attacker performed host reconnaissance, established a reverse shell, collected sensitive information from the compromised system, switched infrastructure to exfiltrate stolen data, and finally conducted internal network reconnaissance by scanning additional systems.

Rather than focusing on individual challenge questions, this report reconstructs the complete attack timeline and explains how each piece of evidence contributed to understanding the attacker's objectives.

Lab Scenario

An engineer observed an abnormal increase in traffic directed toward one of the organization's publicly accessible Ray AI Clusters. Since the Ray server was exposed to the Internet, the unusual traffic raised concerns regarding unauthorized access and possible compromise.

The objective of this investigation was to determine:

- How the attacker gained command execution
- Which systems were involved
- Which commands were executed
- Whether sensitive information was compromised
- How the attacker moved throughout the environment
- The overall impact on the Ray cluster

Investigation Methodology

Instead of searching for answers directly, the investigation followed a structured incident response approach.

Step 1 — Identify Suspicious Hosts

The first objective was to determine which IP addresses were communicating with the Ray server and identify any unusual traffic patterns.

Step 2 — Build the Timeline

Rather than investigating random packets, events were reconstructed chronologically to understand the attacker's actions from initial access through data exfiltration.

Step 3 — Analyze the Ray API

Since the traffic repeatedly referenced `/api/jobs`, `/api/version`, and `/api/cluster_status`, the investigation shifted toward understanding how Ray processes remote jobs.

Step 4 — Correlate Job Submission IDs

Every submitted Ray job generates a unique **submission ID**. Correlating these IDs with network traffic allowed each attacker action to be reconstructed individually.

Step 5 — Analyze HTTP and TCP Streams

HTTP requests provided visibility into submitted commands and API interactions, while TCP stream reconstruction exposed complete conversations and command outputs that were not always visible through HTTP alone.

Step 6 — Extract Indicators of Compromise

Finally, attacker infrastructure, malicious commands, vulnerable services, and affected systems were documented to assess the overall impact of the intrusion.

Technical Investigation

Initial Access

The investigation began by examining HTTP requests targeting the Ray server. Multiple requests were directed toward the **Job Submission API**, immediately suggesting that the attacker was interacting with Ray rather than a traditional web application.

One endpoint appeared repeatedly throughout the capture:

```
POST /api/jobs/
```

Unlike a standard web request, this endpoint allows Ray to create and execute **Jobs** across the cluster. Every submitted job receives a unique **submission ID**, which later became one of the most valuable artifacts during the investigation.

Further analysis revealed that the attacker exploited **CVE-2023-48022**, a vulnerability affecting the Ray Job Submission API. Instead of deploying malware directly, the attacker simply instructed the Ray framework to execute arbitrary Linux commands on their behalf.

This approach is particularly dangerous because the malicious commands appear as legitimate Ray jobs unless proper authentication and monitoring are implemented.

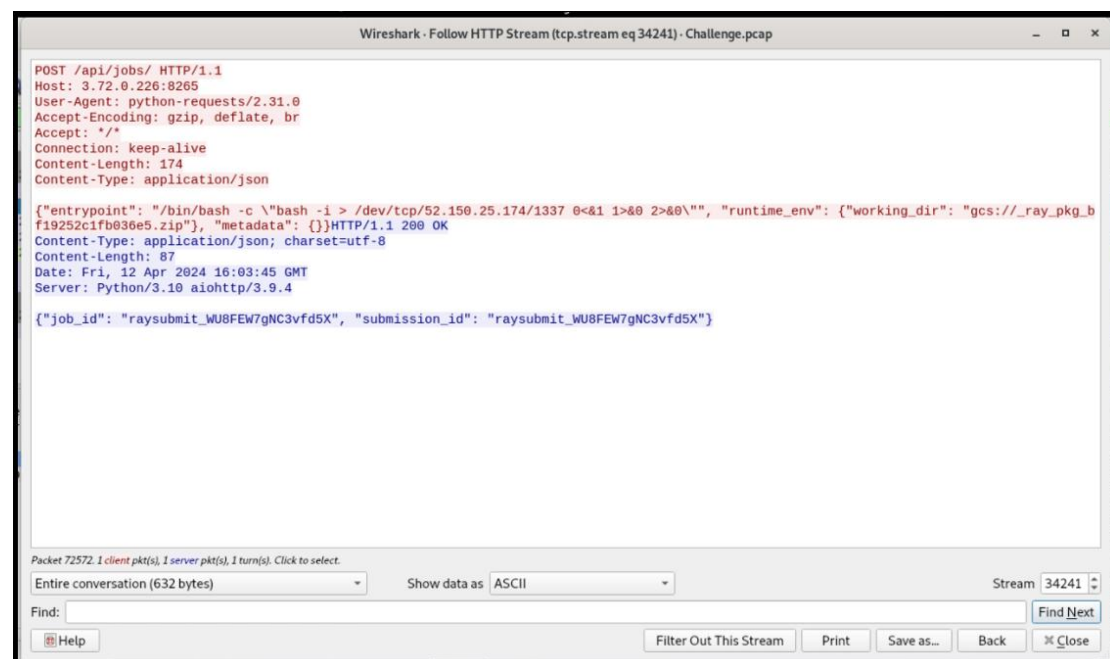


Figure 2 — HTTP POST Request to /api/jobs

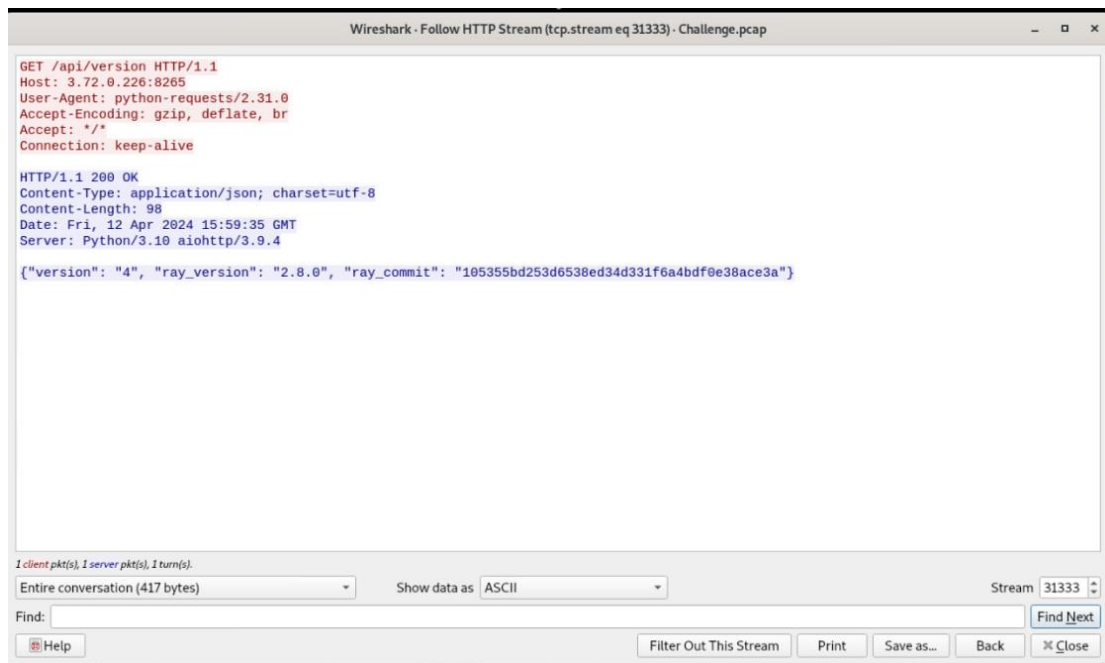


Figure 3 — Ray Version Identification (/api/version)

Key Findings

- Ray Framework identified as the target platform.
- Ray version **2.8.0** confirmed.
- Publicly exposed Job Submission API accessible on **3.72.0.226:8265**.
- Successful exploitation of **CVE-2023-48022** allowed Remote Code Execution.
- Every malicious command generated a unique Ray Job Submission ID, providing valuable evidence throughout the investigation.

Technical Investigation

Command Execution

Following successful exploitation of the exposed Ray Job Submission API, the attacker began executing basic Linux commands to verify that Remote Code Execution (RCE) had been achieved.

The first commands observed were:

```
whoami  
pwdls
```

Although these commands appear harmless, they are commonly used during the reconnaissance phase of an intrusion.

- **whoami** identified the current user executing the commands.
- **pwd** revealed the attacker's current working directory.
- **ls** listed available files and directories, providing an overview of the environment.

Each command was submitted as an individual **Ray Job**, generating a unique **Job Submission ID** that later allowed the investigation to correlate network activity with attacker actions.

Instead of deploying malware immediately, the attacker first focused on understanding the compromised environment. This behavior is commonly observed during the early stages of post-exploitation.

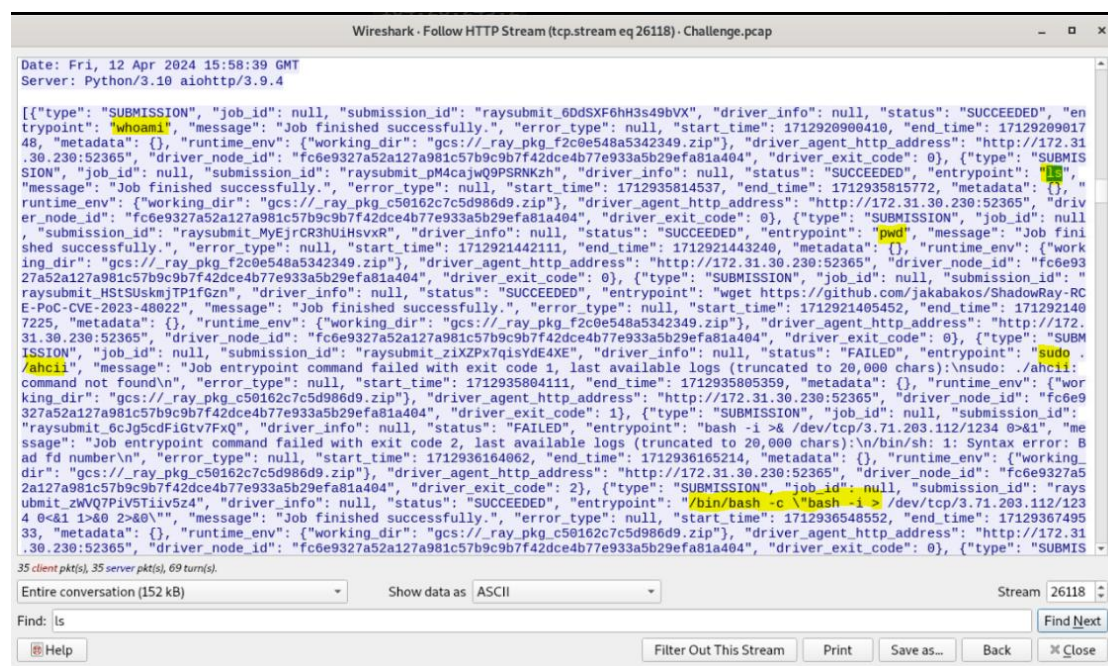


Figure 4 — Job Submission Executing Linux Commands

Reverse Shell Establishment

Submitting individual jobs through the Ray API is effective but inefficient. Every command requires creating a new job, waiting for execution, and retrieving the output.

To obtain a more interactive session, the attacker established a **Bash reverse shell**.

Unlike one-time command execution, a reverse shell provides a continuous interactive connection between the compromised system and the attacker's machine.

This allowed the attacker to execute commands in real time without repeatedly interacting with the Job Submission API.

Initially, only the command itself was visible through HTTP traffic. However, following the TCP stream revealed the command output, exposing the working directory used by the attacker.

This reinforced an important investigation lesson:

HTTP explains **what** command was executed, while TCP stream reconstruction often reveals **what the command actually returned**.

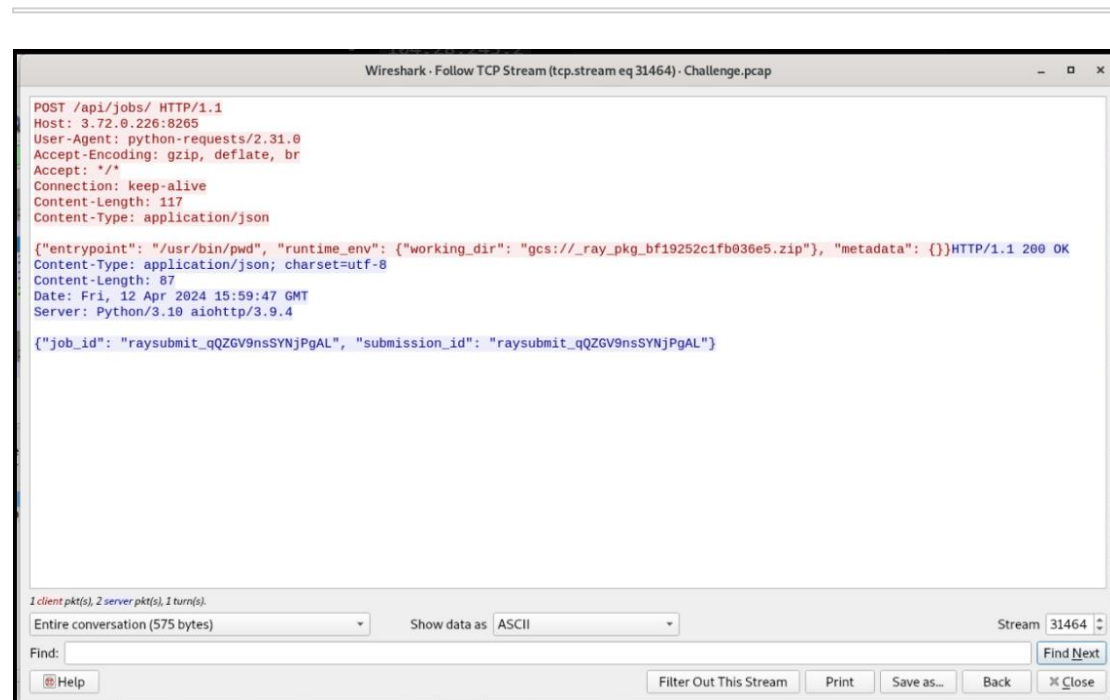


Figure 6 — Working Directory Obtained Through TCP Stream

Data Collection

After confirming access to the compromised system, the attacker searched for valuable information.

A job executing:

```
cat /home/secret.txt
```

was identified during the investigation.

The **cat** command simply displays the contents of a file.

Although this activity exposed sensitive information, it should **not** be classified as exfiltration.

Instead, this represents the **Collection** phase of the intrusion, where the attacker gathers data before transferring it outside the environment.

Understanding this distinction is essential during incident response.

Reading data is **Collection**.

Transmitting data outside the organization is **Exfiltration**.

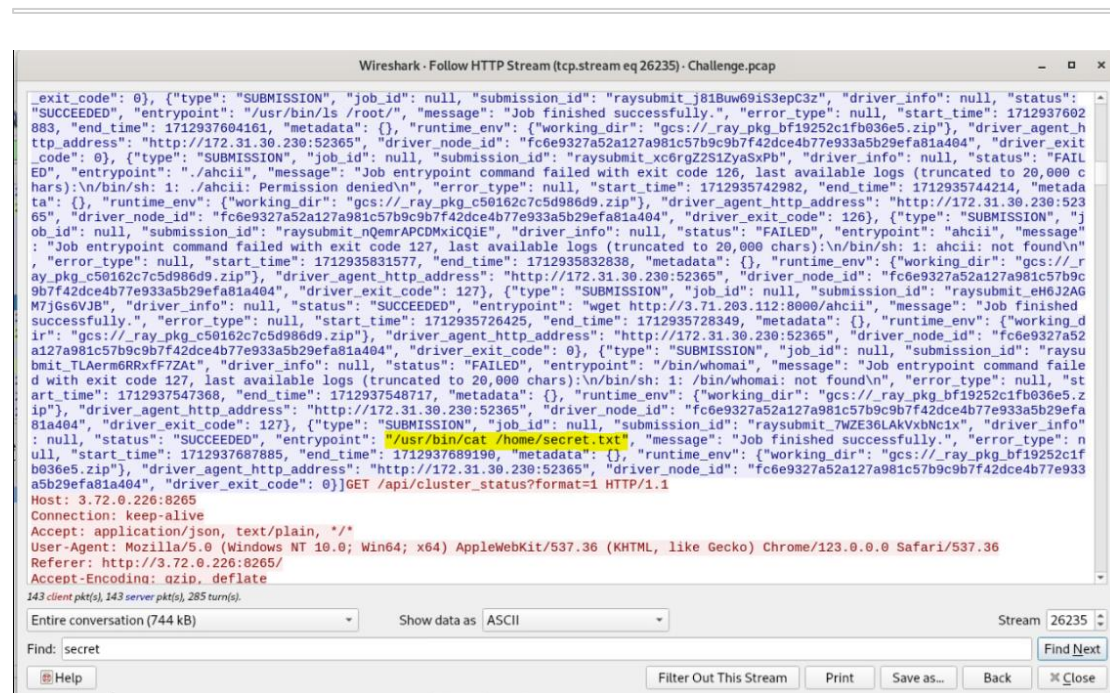


Figure 7 — Reading **secret.txt**

Data Exfiltration

After collecting the contents of the secret file, the attacker changed infrastructure before transmitting the stolen information.

Earlier activity originated primarily from:

104.28.245.2

However, during the exfiltration phase, the attacker switched to:

104.28.213.2

Changing infrastructure during an intrusion is a common technique used to complicate attribution, bypass defensive controls, and reduce the likelihood of detection.

Correlating the new source IP with the malicious Ray Job responsible for reading the secret file confirmed the transition from data collection to data exfiltration.

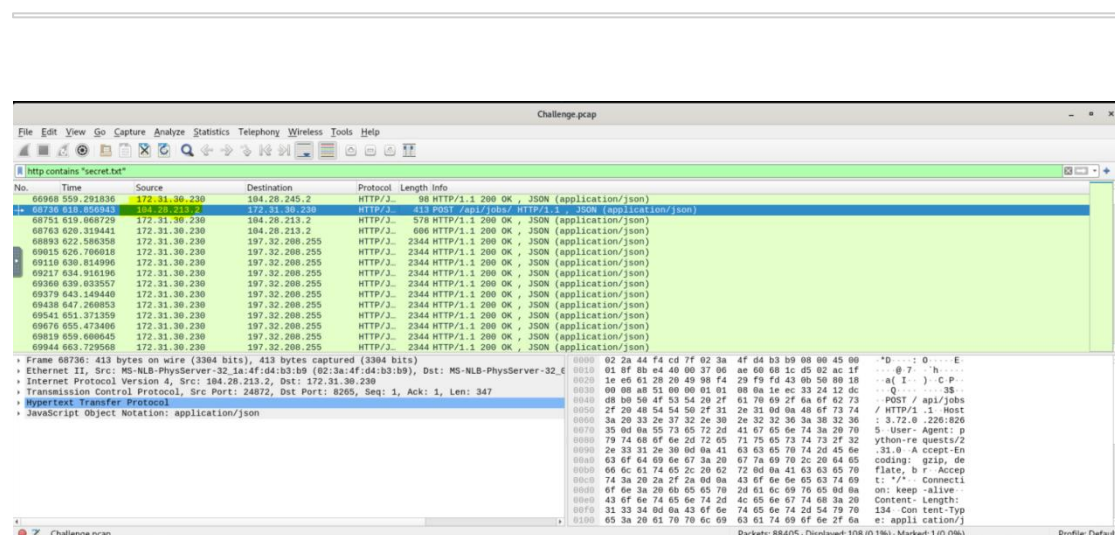


Figure 8 — Exfiltration Activity from New Infrastructure

Internal Network Reconnaissance

One of the most interesting observations occurred near the end of the investigation.

Rather than terminating activity after successfully stealing sensitive information, the attacker launched a large-scale TCP SYN scan against multiple destination ports.

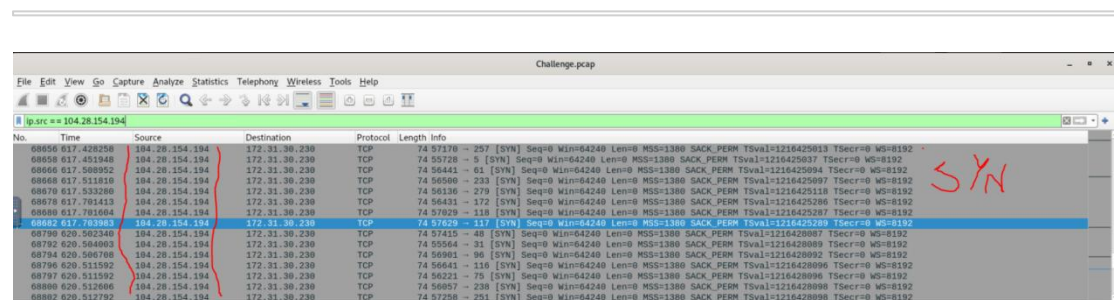
At first glance, it appeared as though the scan might have been part of the initial compromise.

However, reconstructing the timeline revealed that the Ray server had **already been compromised**.

The port scan therefore represented **internal reconnaissance**, allowing the attacker to discover additional services and systems inside the environment for potential lateral movement.

This behavior is frequently observed after attackers establish a stable foothold within a network.

Analysis of Wireshark Endpoints confirmed that the scanning host generated tens of thousands of packets, making it one of the most active systems observed during the investigation.



No.	Time	Source	Destination	Protocol	Length	Info
68556	617.428258	104.28.154.194	172.31.30.230	TCP	74	55728 → 257 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425013 TSecr=0 WS=8192
68558	617.451948	104.28.154.194	172.31.30.230	TCP	74	55728 → 5 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425037 TSecr=0 WS=8192
68560	617.508952	104.28.154.194	172.31.30.230	TCP	74	58442 → 61 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425094 TSecr=0 WS=8192
68568	617.511818	104.28.154.194	172.31.30.230	TCP	74	58980 → 233 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425097 TSecr=0 WS=8192
68570	617.533280	104.28.154.194	172.31.30.230	TCP	74	58130 → 279 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425118 TSecr=0 WS=8192
68578	617.701413	104.28.154.194	172.31.30.230	TCP	74	58432 → 172 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425286 TSecr=0 WS=8192
68580	617.701654	104.28.154.194	172.31.30.230	TCP	74	57829 → 118 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425287 TSecr=0 WS=8192
68582	617.703983	104.28.154.194	172.31.30.230	TCP	74	57629 → 117 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216425289 TSecr=0 WS=8192
68790	620.562340	104.28.154.194	172.31.30.230	TCP	74	57415 → 48 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426007 TSecr=0 WS=8192
68792	620.564803	104.28.154.194	172.31.30.230	TCP	74	53564 → 31 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426089 TSecr=0 WS=8192
68794	620.586708	104.28.154.194	172.31.30.230	TCP	74	56901 → 96 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426992 TSecr=0 WS=8192
68796	620.511592	104.28.154.194	172.31.30.230	TCP	74	56641 → 116 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426998 TSecr=0 WS=8192
68797	620.511592	104.28.154.194	172.31.30.230	TCP	74	56221 → 78 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426998 TSecr=0 WS=8192
68800	620.512606	104.28.154.194	172.31.30.230	TCP	74	56957 → 238 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426998 TSecr=0 WS=8192
68802	620.512792	104.28.154.194	172.31.30.230	TCP	74	57256 → 251 [SYN] Seq=0 Win=64240 Len=0 MSS=1380 SACK_PERM TSval=1216426998 TSecr=0 WS=8192

Figure 9 — TCP SYN Scan

Endpoint Settings

☐ Name resolution

☐ Limit to display filter

Copy

Map

Protocol

☒ Ethernet

☐ FC

☐ FDDI

☐ IEEE 802.11

☐ IEEE 802.15.4

☒ IPv4

☐ IPsec

☐ JXTA

☐ LTP

Filter rule for specific type

Help

Wireshark - Endpoints - Challenge.pcap

Address	Packets	Bytes	Tx Packets	Tx Bytes	Rx Packets	Rx Bytes	Country	City	Latitude	Longitude	AS Number	AS Organization
8.134.178.81	7	518 bytes	1	74 bytes	6	444 bytes						
47.97.80.210	7	518 bytes	1	74 bytes	6	444 bytes						
59.110.5.1	7	518 bytes	1	74 bytes	6	444 bytes						
172.31.0.2	8	1 KB	4	607 bytes	4	436 bytes						
185.242.226.46	8	448 bytes	4	232 bytes	4	216 bytes						
187.71.78.196	9	488 bytes	6	324 bytes	3	162 bytes						
78.128.113.94	12	648 bytes	8	432 bytes	4	216 bytes						
185.254.98.135	12	672 bytes	6	348 bytes	6	324 bytes						
5.188.206.14	24	1 KB	16	864 bytes	8	432 bytes						
52.150.25.174	28	4 KB	15	1 KB	13	2 KB						
52.119.189.171	36	11 KB	20	7 KB	16	4 KB						
79.110.62.62	75	4 KB	50	3 KB	25	1 KB						
169.254.169.123	150	14 KB	78	7 KB	78	7 KB						
169.254.169.254	253	59 KB	125	45 KB	128	14 KB						
212.102.54.50	732	149 KB	423	50 KB	309	99 KB						
3.120.181.43	1,004	106 KB	612	52 KB	392	54 KB						
197.32.208.255	2,646	2 MB	1,342	204 KB	1,304	2 MB						
104.28.213.2	20,971	2 MB	10,639	859 KB	10,332	672 KB						
104.28.245.2	22,508	2 MB	11,410	867 KB	11,098	709 KB						
104.28.154.194	39,598	3 MB	19,799	1 MB	19,799	1 MB						
172.31.30.230	88,362	8 MB	43,660	4 MB	44,702	4 MB						

Figure 10 — Endpoint Statistics

Investigation Takeaways

Several important lessons emerged throughout this investigation:

- Building a chronological timeline proved significantly more effective than searching for isolated indicators.
- Ray Job Submission IDs served as valuable forensic artifacts, allowing commands to be traced throughout the attack.
- TCP stream reconstruction revealed evidence that was not immediately visible through HTTP analysis.
- Distinguishing **Collection** from **Exfiltration** prevented incorrect classification of attacker activity.
- Once the attacker established a foothold, the objective shifted from compromising the Ray server to discovering additional targets within the internal network.

Indicators of Compromise (IOCs)

Throughout the investigation, multiple indicators were identified that helped reconstruct the attack timeline and attribute malicious activity.



Security Analysis Report: Ray AI Cluster Incident

Detailed Incident Indicators and Observed Actions

Indicators of Compromise (IOC)

Indicator	Description
3.72.0.226:8265	Publicly exposed Ray AI Cluster
104.28.245.2	Initial exploitation of the Ray Job Submission API
52.150.25.174:1337	Reverse shell listener
104.28.213.2	Infrastructure used during data exfiltration
104.28.154.194	Internal TCP SYN scanning

Host Indicators

Framework

Ray Framework Version: 2.8.0

Vulnerability

CVE-2023-48022

Commands Observed

- whoami
- pwd
- ls
- cat /home/secret.txt
- bash -i
- wget
- chmod
- ./ahcii

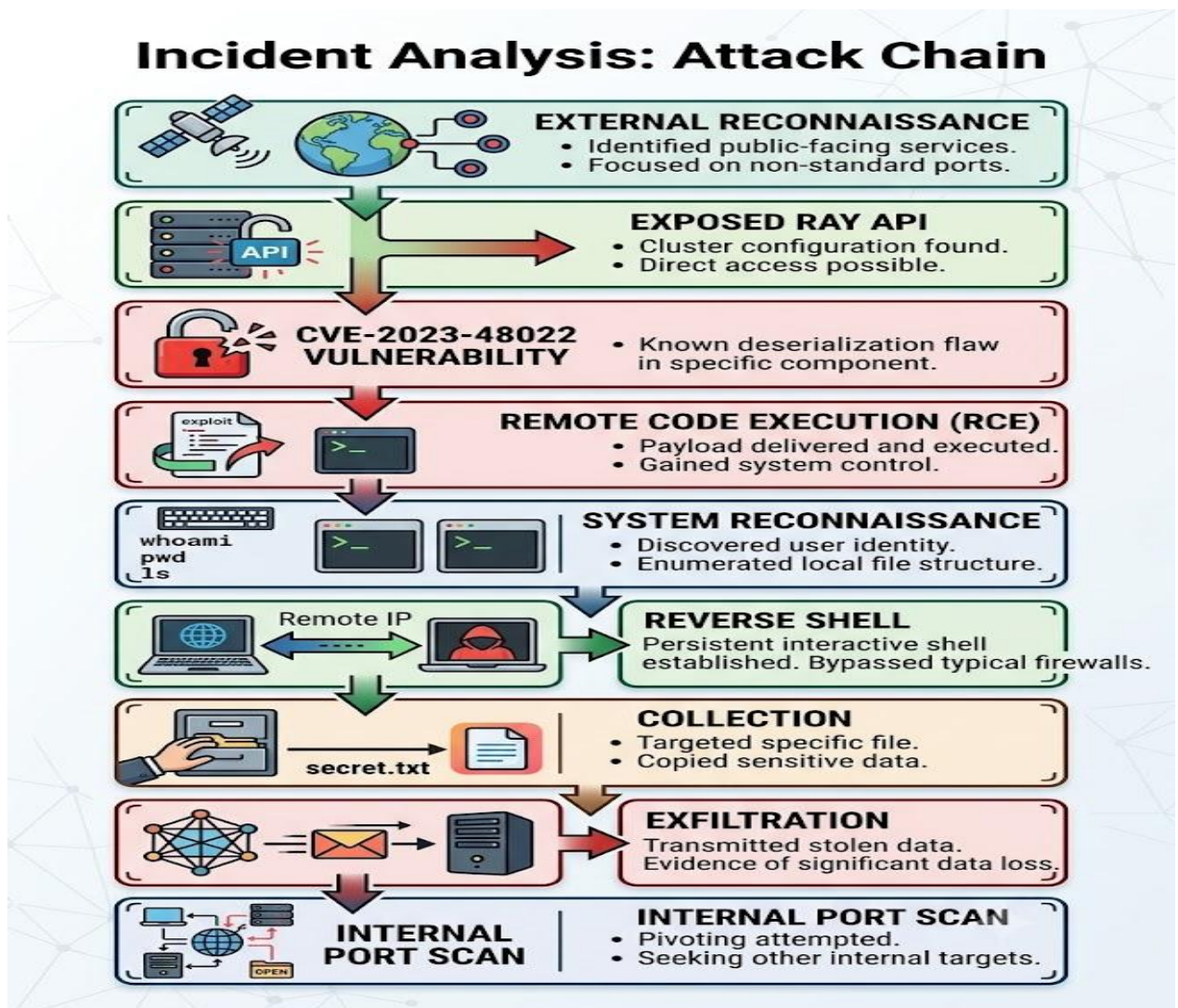
Interesting Files

- secret.txt
- _ray_pkg_XXXXXXXXXX.zip
- ahcii

MITRE ATT&CK Mapping

Tactic (Phase)	Technique ID	Technique Name	Description & Impact
Initial Access	T1190	Exploit Public-Facing Application	Target: Server Software. Outcome: Entry point
Execution	T1059	Command and Scripting Interpreter	Tool: Multiple shells. Outcome: Code run
	T1059.004	Unix Shell / Bash	Specific tool. Outcome: Linux command run
Discovery	T1082	System Information Discovery	Collect: OS, config. Outcome: Target profiling
	T1083	File and Directory Discovery	Find: Key files. Outcome: Data identification
Collection	T1005	Data from Local System	Action: Steal local data. Information theft
Exfiltration	T1041	Exfiltration Over C2 Channel	Path: Command control server. Outcome: Data loss.
	T1046	Network Service Scanning	Find: Other devices. Internal reconnaissance.

Attack Chain



Lessons Learned

Although the attacker relied on relatively simple Linux commands, the combination of an exposed management interface and a vulnerable Ray Job Submission API resulted in complete compromise of the AI cluster.

This investigation highlighted several key lessons:

- Publicly exposed management services should never be accessible without strong authentication.
- Building a chronological timeline is often more effective than searching for isolated indicators.
- Job Submission IDs can serve as valuable forensic artifacts when investigating Ray clusters.
- HTTP analysis explains **how** an attacker interacts with an application, while TCP stream reconstruction often reveals **what actually happened** during those interactions.
- Reading a sensitive file (**Collection**) and transmitting that file (**Exfiltration**) are separate stages of an attack and should be documented independently.
- Once attackers establish a foothold, they rarely stop at a single system. Internal reconnaissance frequently follows successful compromise in preparation for lateral movement.

One of the biggest takeaways from this investigation was realizing that attackers don't always need sophisticated malware. Sometimes they simply abuse legitimate functionality already present within the environment.

Conclusion

This investigation reconstructed the complete compromise of a publicly exposed Ray AI Cluster from initial exploitation through post-exploitation activities.

By abusing **CVE-2023-48022**, the attacker leveraged the Ray Job Submission API to execute arbitrary commands without authentication. After confirming command execution, the attacker established an interactive reverse shell, performed host reconnaissance, collected sensitive information, exfiltrated confidential data, and concluded the intrusion with internal network reconnaissance.

Rather than relying on a single artifact, the investigation combined HTTP analysis, TCP stream reconstruction, Ray Job Submission IDs, endpoint statistics, and attacker infrastructure to build a complete timeline of the attack.

Beyond answering individual challenge questions, this exercise reinforced an important principle of incident response:

The objective isn't simply to identify malicious activity—it's to explain the entire story of the intrusion using evidence.

